Nama : Agus Kariawan

NPM : 065124140

Kelas : O

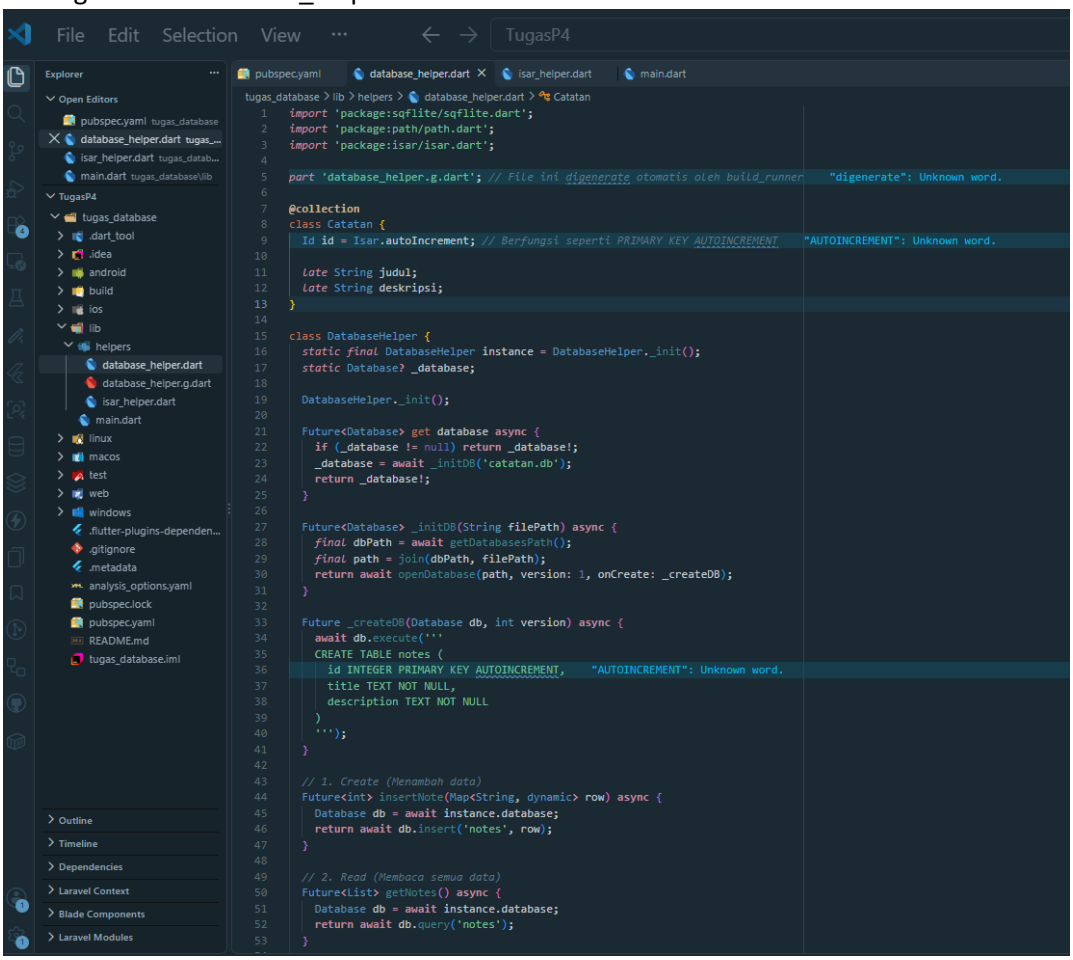
Tugas P4 Mobile Programming

Dokumentasi dan summary hasil uji coba SQFlite dan Storage alternatif lain.

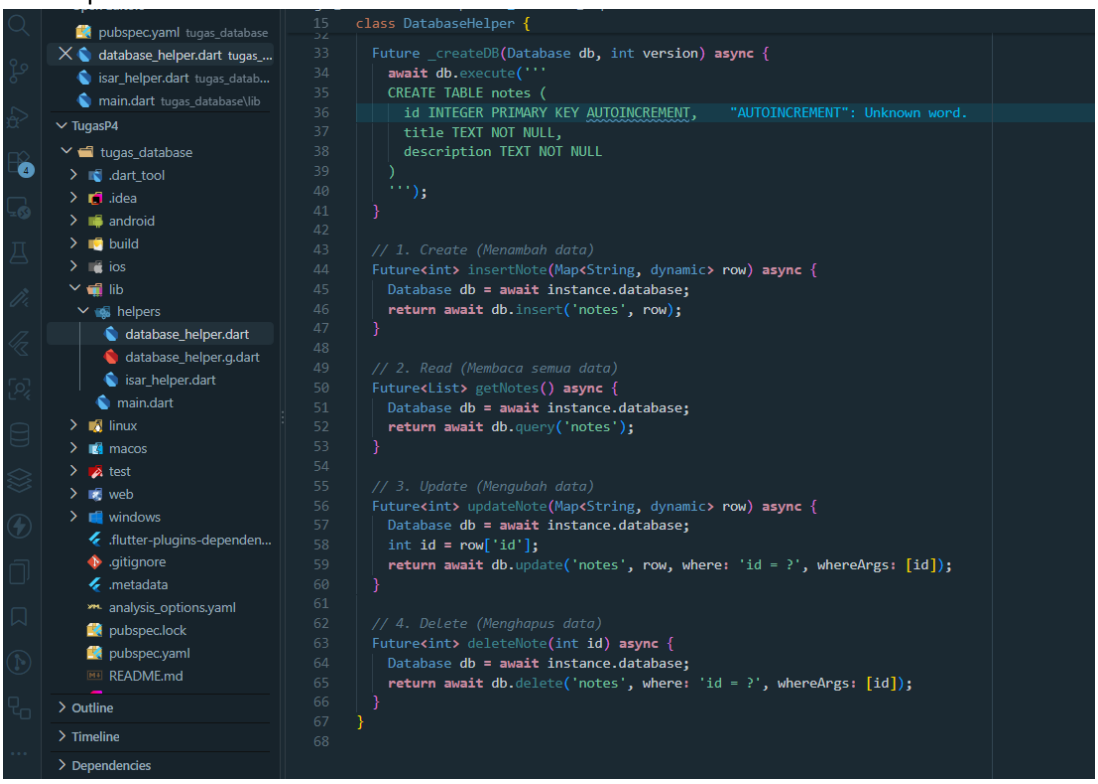
1. Screenshot File pubspec.yaml



Potongan Code database_helper.dart



Code operasi CRUD



```
class DatabaseHelper {
  Future _createDB(Database db, int version) async {
    await db.execute('''
    CREATE TABLE notes (
      id INTEGER PRIMARY KEY AUTOINCREMENT,    "AUTOINCREMENT": Unknown word.
      title TEXT NOT NULL,
      description TEXT NOT NULL
    )
    ''');
  }

  // 1. Create (Menambah data)
  Future<int> insertNote(Map<String, dynamic> row) async {
    Database db = await instance.database;
    return await db.insert('notes', row);
  }

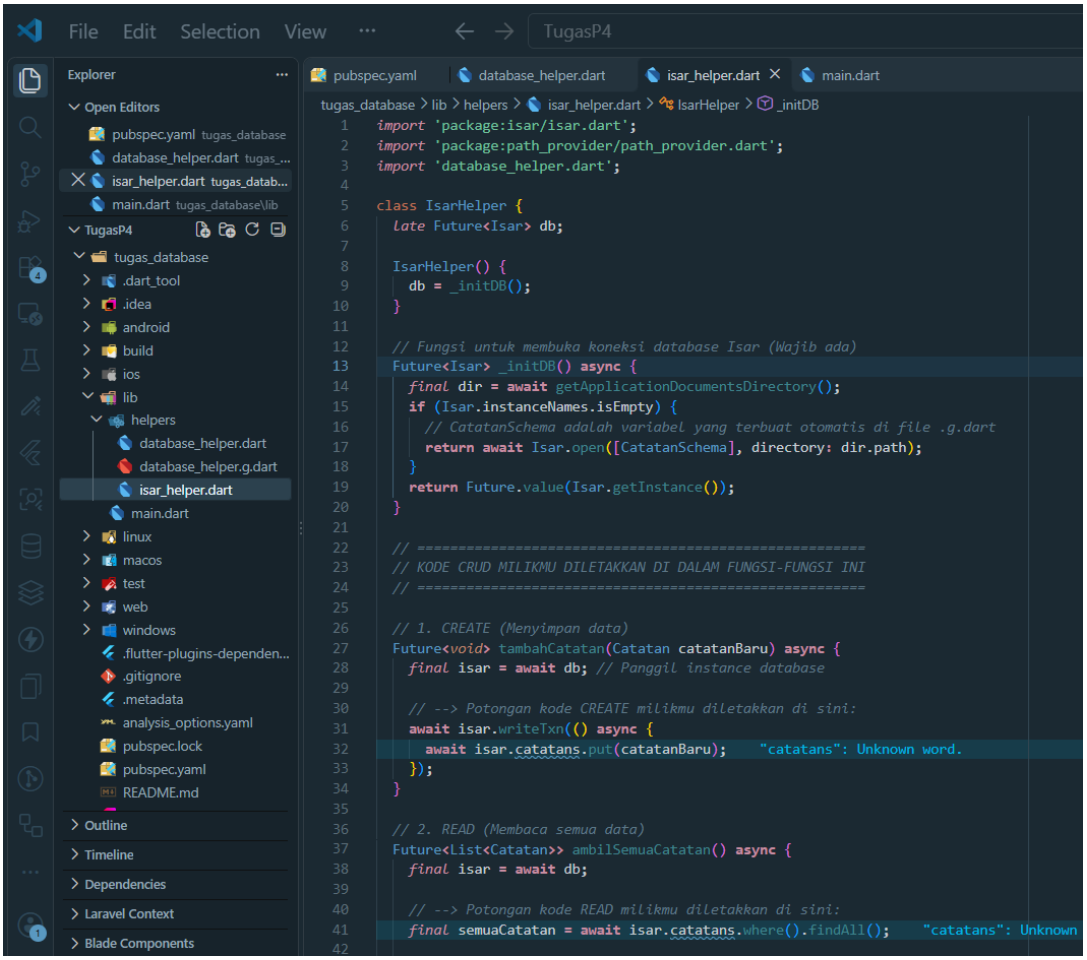
  // 2. Read (Membaca semua data)
  Future<List> getNotes() async {
    Database db = await instance.database;
    return await db.query('notes');
  }

  // 3. Update (Mengubah data)
  Future<int> updateNote(Map<String, dynamic> row) async {
    Database db = await instance.database;
    int id = row['id'];
    return await db.update('notes', row, where: 'id = ?', whereArgs: [id]);
  }

  // 4. Delete (Menghapus data)
  Future<int> deleteNote(int id) async {
    Database db = await instance.database;
    return await db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }
}
```

2. Screenshot Storage Alternatif (Isar)

Code isar_helper.dart



```
import 'package:isar/isar.dart';
import 'package:path_provider/path_provider.dart';
import 'database_helper.dart';

class IsarHelper {
  late Future<Isar> db;

  IsarHelper() {
    db = _initDB();
  }

  // Fungsi untuk membuka koneksi database Isar (Wajib ada)
  Future<Isar> _initDB() async {
    final dir = await getApplicationDocumentsDirectory();
    if (Isar.instanceNames.isEmpty) {
      // CatatanSchema adalah variabel yang terbuat otomatis di file .g.dart
      return await Isar.open([CatatanSchema], directory: dir.path);
    }
    return Future.value(Isar.getInstance());
  }

  // ==========================================================
  // KODE CRUD MILIKMU DILETAKKAN DI DALAM FUNGSI-FUNGSI INI
  // ==========================================================

  // 1. CREATE (Menyimpan data)
  Future<void> tambahCatatan(Catatan catatanBaru) async {
    final isar = await db; // Panggil instance database

    // --> Potongan kode CREATE milikmu diletakkan di sini:
    await isar.writeTxn(() async {
      await isar.catatans.put(catatanBaru);    "catatans": Unknown word.
    });
  }

  // 2. READ (Membaca semua data)
  Future<List<Catatan>> ambilSemuaCatatan() async {
    final isar = await db;

    // --> Potongan kode READ milikmu diletakkan di sini:
    final semuaCatatan = await isar.catatans.where().findAll();    "catatans": Unknown
```

3. Kesumpulan:

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, penggunaan **SQFlite** memberikan fondasi yang solid dalam mengelola *local storage* relasional di Flutter melalui skema tabel terstruktur dan eksekusi query SQL standar untuk fungsi CRUD. Sementara itu, eksplorasi teknologi alternatif NoSQL seperti **Isar** (maupun Hive) membuktikan bahwa penyimpanan data berbasis objek native mampu mempercepat alur pengkodean secara *type-safe* tanpa risiko kesalahan sintaks SQL raw, sekaligus memberikan performa eksekusi yang tinggi.

Dengan demikian, pemilihan teknologi penyimpanan lokal harus disesuaikan dengan kebutuhan arsitektur aplikasi: SQFlite sangat tepat untuk sistem yang membutuhkan relasi data kompleks, sedangkan Isar/Hive menjadi pilihan ideal untuk fleksibilitas pengembangan berbasis objek dan akses data yang cepat